# VMA — Visual Macro Assembler

**Language manual · version 2.1.0**

VMA is the 6502 / 6510 assembler of C64 Visual Assembler — the
language of its Expert mode and of the text its block programs are written as. This manual describes the language;
the editor, the block palette and the other tools of the app are in the C64 Visual Assembler manual.

## Contents


1. Source format
2. Numbers and expressions
3. Output: PRG, BASIC stub and entry point
4. Addressing modes
5. 6502 instructions
6. Directives and macros — reference
7. KickAssembler-style syntax
8. Command line
9. Known limitations


---

## 1. Source format

A VMA source is plain text, one statement per line. Letter case does not matter for mnemonics and directives
( `lda` , `LDA` , `.BYTE` , `.byte` ); label names are case-sensitive.

```
; a comment — so is // a comment
* = $1000                ; ORG: assemble from $1000
start:                   ; a label (the colon is required)
    lda #$06             ; an instruction: mnemonic and operand
    sta $D020
    .byte $01, $02, $FF  ; a directive starts with a dot
loop: dex                ; a label and an instruction on one line
    bne loop
    rts
```

- **Comments** start with `;` or `//` and run to the end of the line (not inside a quoted string).
- **Labels** are written as `name:` — letters, digits and `_`, not starting with a digit. A label can stand on its own line or in front of an instruction. A name starting with a dot ( `.loop:` ) is a **local label** (see Local (dotted) labels); `-` and `+` on their own are **anonymous labels**, referenced by `bne -` / `beq +` (the nearest one before / after).
- **Instructions** are a mnemonic and an operand; indentation is free.

- **Directives** start with a dot ( `.byte` , `.macro` , `.if` …); `* = $xxxx` (or `.pc = $xxxx` ) sets the address.
- **User macros** are called as `:name(args)` , `.invoke name(args)` , `.call name(args)` or `name(args)` .

## 2. Numbers and expressions

| Form | Example | Value |
|---|---|---|
| Hexadecimal | `$D020` , `$0F` | `$` prefix |
| Binary | `%00001111` | `%` prefix |
| Decimal | `255` , `40*25` | no prefix |
| Character | `'A'` | its code ( `$41` ) |
| Current address | `*` | the address of the line (see The `*` program counter) |

In instruction operands write hex numbers with `$` . In the address arguments of the library macros ( `.rawbytes C000, …` , `.sprite_init 0, 1, 80` ) a bare number of one to four digits is read as hex.

**Operators** (in operands, `.const` , `.fill` , `.assert` , `.if` ): `+ - * / %` , `& | ^ ~` , `<< >>` , parentheses, `<expr` (low byte) and `>expr` (high byte), and in conditions `== != < > <= >=` (true = 1, false = 0). `.fill` , `.const` and the like also know `lo()` , `hi()` , `sin()` , `cos()` , `round()` , `min()` , `max()` , `abs()` and `PI` .

```
.const B = 7
    lda #(B+1)*2
    lda #B & 3 | 8
    lda #~B & $FF
    ldx #'A'
    lda #<screen        ; low byte of a label
    lda #>(*+63)        ; high byte of a PC-relative address
```

## 3. Output: PRG, BASIC stub and entry point

VMA writes a C64 **PRG**: a two-byte load address followed by the memory image. Every ORG section and every directive that places data at an address of its own ( `.rawbytes` , `.incbin` , `.sid` …) goes into one image; the gaps between them are zeros.

- **Without an ORG** the program starts at `$0801` , where BASIC starts.
- **BASIC SYS stub** (the default): a `10 SYS nnnn` line is put at `$0801` , so the program starts with `RUN` . A program whose first ORG is `$0801` then starts at `$080D` . Turn the stub off with `--no-basic-stub` ; without it a program at `$0801` is assembled to `$C000` , any other one stays at its first ORG.
- **Entry point:** the SYS line jumps to the start of the code, or to the address named by a first-line comment `; Entry point: $1234` .
- **Code below the BASIC line** (an ORG lower than `$0801` together with the stub): a small loader starts the program and moves it into place, so it still starts with `RUN` .

- A source that begins with its own BASIC SYS line (`.byte $0B, $08, $0A, $00, $9E, …` at `$0801`) is emitted as it is, without a second stub.

From the PRG the command line also makes a **cartridge** and a **disk image** (new in 2.1.0, see 8. Command line):

- `--crt FILE` — a Magic Desk cartridge (CRT type 19, 8 × 8K banks, at most 65 408 bytes of program). A loader in bank 0 initialises the KERNAL, runs a copy routine from RAM at `$0100–$013B` that moves the program to its load address (switching banks through `$DE00`), turns the cartridge off and jumps to the entry point. Keep the program out of `$0100–$013B` and the zero page bytes `$02–$04` / `$FB–$FE` the loader uses.
- `--d64 [FILE]` — a 1541 disk image (35 tracks) with the PRG named after it, plus the `--add` files, laid out as the 1541 DOS writes them; no c1541 needed.

---

## 4. Addressing modes

Each 6502 instruction supports one or more addressing modes. VMA picks the mode from how the operand is written.

| Mode | Label | Example | Description |
|---|---|---|---|
| **implied** | Implied | `NOP` | No operand; the instruction is self-contained |
| **immediate** | Immediate | `LDA #$FF` | Inline constant; the assembler adds `#` automatically |
| **zeroPage** | Zero page | `LDA $10` | Single byte address in page zero (0–255) |
| **zeroPageX** | Zero page,X | `LDA $10,X` | Zero page address + X register offset (result wraps in page 0) |
| **zeroPageY** | Zero page,Y | `LDX $FB,Y` | Zero page address + Y register offset |
| **absolute** | Absolute | `LDA $0400` | Full 16-bit memory address |
| **absoluteX** | Absolute,X | `LDA $0400,X` | 16-bit address + X register offset |
| **absoluteY** | Absolute,Y | `LDA $0400,Y` | 16-bit address + Y register offset |
| **relative** | Relative/Label | `BNE loop` | For branch instructions; enter a label name or target address |
| **indirectX** | Indirect,X | `LDA ($FB,X)` | Zero page indexed indirect (operand = zero page address, 1 byte) |
| **indirectY** | Indirect,Y | `LDA ($FB),Y` | Zero page indirect indexed (operand = zero page address, 1 byte) |
| **indirect** | Indirect | `JMP ($0100)` | Indirect; only usable with JMP |

### Constants in the zero page

An operand made of constants only (`.const`, `name = value`, a `.var`) that fits in a byte takes the instruction's **zero-page** mode, exactly as the number would: after `.const zp = $FB`, `lda zp` assembles to `A5 FB` (2 bytes, 3 cycles) and `lda zp,x` to `B5 FB`. A mode the instruction has no zero-page form of stays absolute (`lda zp,y` → `B9 FB 00`). A label is not narrowed — its address is known only after the layout — and `lda.abs zp` forces the absolute mode. A number keeps the width it is written with: `$FB` is zero page, `$00FB` absolute. *(New in 2.1.)*

### Label expressions as operands

Any operand that accepts an address or immediate value also accepts a **constant name** (a `.const` or a label) directly. Additionally, you can use **label+offset** or **label−offset** expressions to reference an address relative to a named constant:

| Syntax | Example | Description |
|---|---|---|
| `label` | `STA screen_ram,X` | Resolves to the label/constant value |
| `label+$hex` | `STA screen_ram+$0100,X` | Label address plus a hex offset |
| `label+decimal` | `STA screen_ram+256,X` | Label address plus a decimal offset |
| `label-$hex` | `LDA table-$10` | Label address minus a hex offset |
| `$hex+expr` | `LDA $1234+1`, `STA $0400+40*2,X` | Any expression of numbers, labels and constants *(2.1)* |
| `(label)` | `JMP (vector)`, `JMP (table+2)` | Indirect jump through a label or an expression *(2.1)* |
| `#<label` | `LDA #<screen_ram` | Low byte of the label address |
| `#>label` | `LDA #>screen_ram` | High byte of the label address |
| `*` | `BNE *` | Current program counter (the instruction's own address); branches with `*` generate an infinite self-loop (offset `$FE`) |

**Example — clear two screen pages using a CONST:**

```
; .CONST screen_ram = $0400
    LDX #$00
clear:
    STA screen_ram,X
    STA screen_ram+$0100,X
    DEX
    BNE clear
```

### The `*` program counter in expressions

`*` is no longer limited to being the whole operand — it can appear **anywhere inside an operand expression**, and stands for the address of the instruction it is written on. It is resolved at assembly time against that instruction's real address, so no label is needed for short relative jumps or PC-relative data reads.

| Syntax | Example | Meaning |
|---|---|---|
| `*` | `BNE *` | Branch to self (infinite loop, offset `$FE`) |
| `*-n` / `*+n` | `BNE *-5`, `BEQ *+4` | Branch relative to the current PC by *n* bytes |
| `JMP *+n` | `JMP *+20` | Absolute jump computed from the current PC |
| `#<*` / `#>*` | `LDA #<*`, `LDA #>*` | Low / high byte of the current PC |

| Syntax | Example | Meaning |
| --- | --- | --- |
| `#>(*+n)` | `LDA #>(*+63)` | Low / high byte of a PC-relative address |

**PC vs. multiplication.** `*` is treated as the program counter only when it sits in *value position* — at the start of the expression, or right after an operator, `(`, `,`, `<`, `>` or whitespace. A `*` that follows a number, `)` or an identifier is ordinary multiplication, so `LDA table*2` and `CONST_A*4` are unchanged.

**Where it works.** Any operand that already accepts an expression: branch targets, `JMP` / `JSR`, `LDA` / `STA` /… absolute and indexed, immediate low/high-byte operators, and the `.assert` expression. `*` never changes an instruction's size, so it is safe in every addressing mode.

### Local (dotted) labels

A label whose name starts with a dot — `.loop`, `.skip`, `.done` — is a **local label**. It belongs to the scope of the nearest preceding **global** (non-dotted) label, and internally becomes `<global>.<name>`. Two local labels with the same short name under different global labels do **not** collide.

```
DrawSprite:
    LDX #0
.loop:                ; == DrawSprite.loop
    LDA SpriteData,X
    STA $2000,X
    INX
    CPX #63
    BNE .loop         ; resolves within DrawSprite
    RTS

ClearScreen:
    LDX #0
.loop:                ; == ClearScreen.loop — no clash
    STA $0400,X
    INX
    BNE .loop
    RTS
```

- Inside a scope, reference a local label as `.name`.
- From another scope, reference it explicitly as `Global.name` (e.g. `JMP ClearScreen.loop`).
- A `.name` written before any global label stays a plain top-level `.name`.

### Self-modifying-code operand labels

Prefix an instruction's operand with `label:` to place a label on the **operand byte** rather than the opcode. The instruction assembles from whatever follows the colon.

```
setup:
    LDA value:#$00     ; 'value' -> address of the #$00 operand byte
    ...
patch:
    LDA #new
    STA value          ; writes the operand byte directly — classic SMC
```

`value` points at `<instruction address> + 1` (the first operand byte), for every addressing mode. This replaces the older `STA instruction+1` / `instruction: LDA #$00` pattern.

---

## 5. 6502 instructions

### Data Movement

| Mnemonic | Description | Modes |
|---|---|---|
| `LDA` | Load Accumulator | immediate, zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY |
| `LDX` | Load X register | immediate, zeroPage, zeroPageY, absolute, absoluteY |
| `LDY` | Load Y register | immediate, zeroPage, absolute, absoluteX |
| `STA` | Store Accumulator | zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY |
| `STX` | Store X register | zeroPage, zeroPageY, absolute |
| `STY` | Store Y register | zeroPage, absolute |

### Arithmetic

| Mnemonic | Description | Notes |
|---|---|---|
| `ADC` | Add with carry | Set carry with `SEC` before use in most cases |
| `SBC` | Subtract with carry | Set carry with `SEC` before subtraction |
| `INC` | Increment memory | — |
| `DEC` | Decrement memory | — |
| `CMP` | Compare with A | Sets flags; does not modify A |
| `CPX` | Compare with X | — |
| `CPY` | Compare with Y | — |

### Logic

| Mnemonic | Description |
|---|---|
| `AND` | Logical AND with Accumulator |
| `ORA` | Logical OR with Accumulator |
| `EOR` | Exclusive OR with Accumulator |
| `BIT` | Test bits in memory against A (sets N, V, Z flags) |

Jumps & Branches

| Mnemonic | Description |
|---|---|
| `JMP` | Unconditional jump (absolute or indirect) |
| `JSR` | Jump to subroutine (saves return address on stack) |
| `RTS` | Return from subroutine |
| `RTI` | Return from interrupt |
| `BNE` | Branch if Not Equal (Z=0) |
| `BEQ` | Branch if Equal (Z=1) |
| `BCC` | Branch if Carry Clear (C=0) |
| `BCS` | Branch if Carry Set (C=1) |
| `BMI` | Branch if Minus (N=1) |
| `BPL` | Branch if Plus (N=0) |
| `BVC` | Branch if Overflow Clear (V=0) |
| `BVS` | Branch if Overflow Set (V=1) |

**LBNE / LBEQ / … (Long branches)**

Eight pseudo-ops behave
like conditional branches but reach **any address**, not just −128…+127. Each one assembles
to an inverted branch that skips a 3-byte `JMP` — always **5 bytes**:

```
LBEQ done      ; assembles to:   BNE *+3   ($D0 $03)
               ;                 JMP done  ($4C lo hi)
```

| Long op | Condition | Emitted as |
|---|---|---|
| `LBNE` | not equal (Z=0) | `BEQ *+3 / JMP target` |
| `LBEQ` | equal (Z=1) | `BNE *+3 / JMP target` |
| `LBCC` | carry clear (C=0) | `BCS *+3 / JMP target` |

| Long op | Condition | Emitted as |
|---|---|---|
| `LBCS` | carry set (C=1) | `BCC *+3 / JMP target` |
| `LBMI` | minus (N=1) | `BPL *+3 / JMP target` |
| `LBPL` | plus (N=0) | `BMI *+3 / JMP target` |
| `LBVC` | overflow clear (V=0) | `BVS *+3 / JMP target` |
| `LBVS` | overflow set (V=1) | `BVC *+3 / JMP target` |

- Operand: a label, a `*`-expression, or a literal address — same as a normal branch target.
- Cost: 5 bytes and 1 extra cycle on the taken path vs. a short branch. There is no automatic promotion of a short branch — you choose `LBxx` explicitly.
- When a plain branch ( `BNE` , `BEQ` , …) is out of range, the compiler error now names the exact overshoot and suggests the matching `LBxx` .

### Register Operations

| Mnemonic | Description |
|---|---|
| `TAX` | Transfer A → X |
| `TAY` | Transfer A → Y |
| `TXA` | Transfer X → A |
| `TYA` | Transfer Y → A |
| `TSX` | Transfer Stack pointer → X |
| `TXS` | Transfer X → Stack pointer |
| `INX` | Increment X |
| `DEX` | Decrement X |
| `INY` | Increment Y |
| `DEY` | Decrement Y |

### Shift & Rotate

| Mnemonic | Description |
|---|---|
| `ASL` | Arithmetic Shift Left |
| `LSR` | Logical Shift Right |
| `ROL` | Rotate Left through Carry |
| `ROR` | Rotate Right through Carry |

### Stack

| Mnemonic | Description |
|---|---|
| `PHA` | Push Accumulator onto stack |
| `PHP` | Push Processor status onto stack |
| `PLA` | Pull Accumulator from stack |
| `PLP` | Pull Processor status from stack |

### System / Flags

| Mnemonic | Description |
|---|---|
| `CLC` | Clear Carry flag |
| `CLD` | Clear Decimal mode |
| `CLI` | Clear Interrupt disable |
| `CLV` | Clear Overflow flag |
| `SEC` | Set Carry flag |
| `SED` | Set Decimal mode |
| `SEI` | Set Interrupt disable |
| `NOP` | No operation |
| `BRK` | Force break / software interrupt |

### Illegal / Undocumented Instructions

These are supported for advanced use. Use with care — behavior may differ between chips.

`LAX`, `SAX`, `DCP`, `ISC`, `SLO`, `RLA`, `SRE`, `RRA`, `ANC`, `ALR`, `ARR`, `AXS`, and the `NOP` forms that read an operand: `NOP #$nn` ($80), `NOP $nn` ($04), `NOP $nn,X` ($14), `NOP $nnnn` ($0C), `NOP $nnnn,X` ($1C)

---

## 6. Directives and macros — reference

Directives control the assembly (labels, data, origin, conditions, user macros); the library macros do common C64 tasks in one line — instead of writing 10–20 instructions by hand, you write one line and the assembler generates the code. Think of them as built-in subroutines.

---

### LABEL

Like a **line number in BASIC** — but with a name instead of a number. Jump targets for `JMP`, `JSR`, `BNE`, etc.

| Argument | Description |
|---|---|
| Label name | Identifier used in `JMP`, `JSR`, `BNE`, etc. |

**Syntax:**

```
loop:
```

**Generated code:**

```
loop:  ; $0820
```

The current address is shown as a comment. Labels have **0 byte size**.

---

COMMENT

Like **REM in BASIC** — a note for yourself that the assembler ignores completely.

**Syntax:**

```
; Your comment text here
```

**Generated code:**

```
; Your comment text here
```

---

BYTE

Like **DATA in BASIC** — stores a list of raw byte values inline in the program.

| Argument | Description |
| --- | --- |
| Operand | Comma-separated byte values (e.g. `$01, $02, $FF` or `1, 2, 255`) |

**Syntax:**

```
.byte $01, $02, $FF
```

**Generated code:**

```
    .byte $01, $02, $FF
```

**Lo/hi byte label references:** BYTE accepts KickAssembler / ca65 style `<label` (low byte) and `>label` (high byte) tokens alongside numeric values. The assembler resolves the label address at compile time and inserts the appropriate byte. Example:

```
    .byte <frame_0, >frame_0, <frame_1, >frame_1
```

This stores the low byte of `frame_0`'s address, then the high byte, then the same for `frame_1`. Useful for building jump tables and address lists.

**Size:** Number of bytes in the list.

---

WORD

Like **DATA in BASIC but for 16-bit numbers**. Each value is stored as two bytes (low byte first, then high byte — 6502 little-endian order).

| Argument | Description |
|---|---|
| Operand | Comma-separated 16-bit values (e.g. `$0400, $C000`) |

**Syntax:**

```
.word $0400, $C000
```

**Generated code:**

```
    .word $0400, $C000
```

**Size:** 2 bytes per word.

---

FILL

Like `FOR I=1 TO N : POKE addr+I, val : NEXT` — fills a block of memory with the same byte, in one line. Great for clearing areas or pre-filling tables.

| Argument | Description |
|---|---|
| Operand | `count,value` — e.g. `256,0` fills 256 bytes with zero |

**Syntax:**

```
.fill 256, $00
```

**Generated code:**

```
    .fill 256, $00
```

**Expression syntax:** Both `count` and `value` accept arithmetic expressions. You can reference CONST names, use hex/binary literals, and call built-in math functions:

| Expression | Meaning |
|---|---|
| `TILE_COUNT, $00` | count from a CONST, value hex literal |
| `40*25, 0` | inline multiplication |
| `round(sin(PI/4)*255), $80` | trigonometry |

**Built-in functions:** `sin()`, `cos()`, `round()`, `max(a,b)`, `min(a,b)`, `abs()`, constant `PI`

Operators: `+ - * /` Literals: `$FF` (hex), `%10110000` (binary) Low/high byte: `lo(expr)`, `hi(expr)`

**Size:** The count value in bytes.

---

ALIGN

Slides the current address forward to the next clean boundary by inserting zero-padding bytes. The C64 requires sprite data to start on a 64-byte boundary — `ALIGN 64` handles that automatically.

| Argument | Description |
|---|---|
| Boundary | Alignment value — e.g. `64` (sprite boundary), `256` (page), `$2000` (bitmap) |

**Syntax:**

```
.align 64
.align $2000
```

**Generated code:**

```
; ALIGN 64 → $0840 (12 bytes padding)
```

**Size:** Dynamic — depends on the current program counter position.

> **Tip:** Use `ALIGN 64` before sprite data, `ALIGN 256` to ensure page-aligned tables.

---

TEXT

Like **PRINT AT** — writes text directly to the C64 screen at a given column and row, without using the KERNAL. It generates one LDA/STA pair per character, targeting screen RAM at `$0400`.

| Argument | Description |
|---|---|
| Text | The string to display |
| X | Column (0–39) |
| Y | Row (0–24) |
| Label (optional) | Assigns a label pointing to the computed screen address |
| `lower` | Optional flag — see below |

**Charset modes:**

The C64 has two character sets selectable at runtime:

| Mode | $D018 bit 1 | Uppercase input | Lowercase input |
|---|---|---|---|
| **Uppercase/graphics** (default) | 0 | `A` – `Z` → screen codes $01–$1A ✓ | treated as uppercase too |
| **Lowercase/uppercase** (after CHARSET lower) | 1 | `A` – `Z` → $01–$1A (uppercase) | `a` – `z` → $41–$5A (lowercase) ✓ |

- **Uppercase charset (default):** Type what you want to see in uppercase. `"HELLO"` displays as `HELLO`. Lowercase input is mapped to uppercase screen codes.
- **Lowercase charset (`lower`):** Type the exact case you want to see. `"hello"` → lowercase display, `"HELLO"` → uppercase display. Requires a runtime charset switch before the screen is written (use the **CHARSET lower** macro).

**Generated code (uppercase mode, `"HELLO"`):**

```
LDA #$08      ; 'H' screen code $08
STA $0400
LDA #$05      ; 'E' screen code $05
STA $0401
...
```

**Syntax:**

```
.text 0, 2, "HELLO"           ; uppercase charset (default)
.text 0, 2, "hello", lower    ; lowercase charset
```

Characters are encoded as **screen codes** (not PETSCII). **Size:** `text.length × 5` bytes (LDA + STA per character).

---

STRING

Like **POKEing a string** into any memory address at runtime. Generates LDA/STA pairs that copy each character's screen code to consecutive addresses.

| Argument | Description |
|---|---|
| Text | The string to write |
| Address | Target memory address — `$C000` hex or a **label name** |
| Label (optional) | Assigns a label pointing to the target address |
| Shift | Hex value (00–FF) added to each screen code byte (e.g. `$80` = reverse video) |
| `lower` | Optional flag — same semantics as TEXT (see TEXT section) |

**Syntax:**

```
.string $C000, "HELLO"                 ; uppercase charset (default)
.string $C000, "hello", lower          ; lowercase charset
.string $C000, "HELLO", 80             ; with shift (reverse video)
.string $C000, "hello", 80, lower      ; shift + lowercase
.string $C000, "HELLO" :my_string      ; with macroLabel
```

**Generated code:**

```
    LDA #$08      ; 'H' screen code
    STA $C000
    LDA #$05      ; 'E' screen code
    STA $C001
    ...
```

Characters are encoded as **screen codes** (not PETSCII). The optional **Shift** value is added to every byte, e.g. `$80` for reverse video. **Size:** `text.length × 5` bytes (one LDA + one STA per character).

---

DATA

Like a **POKE loop** — writes a list of raw bytes to a memory address at runtime, one LDA/STA pair per byte.

| Argument | Description |
|---|---|
| Bytes | Comma-separated byte values |
| Address | Target memory address — `$C000` hex or a **label name** |
| Label (optional) | Assigns a label pointing to the target address |

**Syntax:**

```
.data $C000, $01, $02, $03          ; hex address
.data my_buf, $01, $02, $03         ; label address
.data $C000, $01, $02, $03 :mydata  ; with macroLabel
```

**Generated code:**

```
LDA #$01
STA $C000
LDA #$02
STA $C001
...
```

**Size:** `byte_count × 5` bytes (one LDA + one STA per byte).

> **2.1:** the bytes may name labels: `.data $2010, <target, >target` writes the address of `target`, also of a label defined later. The `:name` label of the macro may point at a target address that is itself a label.

---

RAWBYTES

Like **DATA that loads directly into memory** — no runtime code at all. The bytes are present from the moment the PRG loads, before your code even starts. Use this for sprite data, level maps, lookup tables, anything that just needs to be at a specific address.

| Argument | Description |
|---|---|
| Bytes | Comma-separated byte values |
| Address | Target memory address — `$C000` hex or a **label name** |
| Label (optional) | Assigns a label pointing to the target address |

**Syntax:**

```
.rawbytes $C000, $00, $00, $00      ; hex address
.rawbytes sprite_data, $00, $00     ; label address
.rawbytes $0C50, $00, $00 :nev      ; with macroLabel — other code can use LDA nev,X
```

**Size in code:** 0 bytes. The data is placed at the given address in the output.

> **DATA vs RAWBYTES:** DATA generates LDA/STA code that copies bytes at runtime (slower, but works if the data needs to be dynamic). RAWBYTES just places the bytes directly — no code, instant, zero cost.

> **2.1:** the bytes may name labels ( `.rawbytes $2000, <target, >target` — a pointer table); the `:name` label may point at a label as the target address, also one defined later. As a macro's buffer argument, the bytes take their own space in the buffers after the program.

---

RAWTEXT

Like RAWBYTES but for text — encodes the string as screen codes and places the bytes at a fixed address with **no runtime code**. The text is ready in memory the instant the PRG loads.

| Argument | Description |
|---|---|
| Text | String to encode |
| Address | Target memory address — `$C000` hex or a **label name** |
| Label (optional) | Assigns a label pointing to the target address |
| Shift | Hex value (00–FF) added to each screen code byte (e.g. `$80` = reverse video) |
| `lower` | Optional flag — same semantics as TEXT (see TEXT section) |

**Syntax:**

```
.rawtext $C000, "HELLO"                ; uppercase charset (default)
.rawtext $C000, "hello", lower         ; lowercase charset
.rawtext $C000, "HELLO", 80            ; with shift (reverse video)
.rawtext $C000, "hello", 80, lower     ; shift + lowercase
.rawtext $0400, "HELLO" :my_text       ; with macroLabel
```

**Generated code:**

```
; .rawtext "HELLO" -> $C000
; $C000
    .byte $08, $05, $0C, $0C, $0F   ; H E L L O (uppercase screen codes)

; .rawtext "hello", lower -> $C000
; $C000
    .byte $48, $45, $4C, $4C, $4F   ; h e l l o (lowercase screen codes $41–$5A range)
```

**Size in code:** 0 bytes. The data is placed at the given address in the output.

> **STRING vs RAWTEXT:** STRING generates LDA/STA code that copies the text at runtime. RAWTEXT bakes the bytes into the PRG at load time — no code, no waiting.

---

PETSCII

Like **RAWBYTES but for KERNAL output** — encodes the string as PETSCII bytes (compatible with CHROUT at `$FFD2`) and places them at a fixed address with no runtime code. Use this when you want to print characters via `JSR $FFD2` in a loop, and note that `.print` uses the same encoder and `lower` flag.

> **PETSCII vs screen codes:** PETSCII and screen codes are two different encodings. Screen code `$01` = letter A; PETSCII `$41` = letter A (via CHROUT). Use PETSCII only when printing through the KERNAL; use TEXT/STRING/RAWTEXT for writing directly to screen RAM.

| Argument | Description |
|---|---|
| Text | String to encode as PETSCII bytes |
| Address | Target memory address — `$C000` hex or a **label name** |
| Label (optional) | Assigns a label pointing to the target address |
| `lower` | Optional flag — see below |

**Charset modes:**

| Mode | Uppercase input ( `A` – `Z` ) | Lowercase input ( `a` – `z` ) |
|---|---|---|
| **Uppercase (default)** | `$41` – `$5A` (PETSCII uppercase via CHROUT) | also mapped to `$41` – `$5A` |
| **Lowercase ( `lower` )** | Alphabetic letters are remapped so the visible case stays consistent on the lowercase/uppercase charset | Same rule |

**Syntax:**

```
.petscii $C000, "HELLO"              ; uppercase PETSCII (default)
.petscii $C000, "hello", lower       ; lowercase PETSCII ($61–$7A)
.petscii $C000, "HELLO", null        ; with null terminator
.petscii $C000, "hello", lower, null ; lowercase + null terminator
.petscii $C000, "HELLO" :my_msg      ; with macroLabel
```

**Generated bytes (uppercase, `"HELLO"`):**

```
; $C000
    .byte $48, $45, $4C, $4C, $4F   ; H E L L O (PETSCII $41–$5A range)
```

**Size in code:** 0 bytes. The data is placed at the target address as a deferred data section (like RAWBYTES).

**Null terminator:** add the `null` flag to append a `$00` byte after the text. Ideal for null-terminated loops:

```
    LDX #$00
loop:
    LDA msg,X
```

```
    BEQ done        ; $00 stops the loop
    JSR $FFD2
    INX
    BNE loop
done:
    RTS
```

**Encoding rules:**

| Input | Uppercase mode | Lowercase mode |
|---|---|---|
| `A` – `Z` | `$41` – `$5A` | `$61` – `$7A` |
| `a` – `z` | `$41` – `$5A` (forced uppercase) | `$41` – `$5A` |
| Space, digits, punctuation (32–126) | as-is | as-is |
| Newline | `$0D` (RETURN) | `$0D` |
| Other | `$20` (space) | `$20` |

> **Tip:** Use PETSCII for data that will be output via CHROUT ( `$FFD2` ). For writing directly to screen RAM, use STRING or RAWTEXT instead.

---

CHARSET

Switches the VIC-II character ROM between uppercase/graphics mode (C64 default) and lowercase/uppercase mode, by modifying bit 1 of `$D018` at runtime.

| Argument | Description |
|---|---|
| Mode | **Lowercase** — enables the lowercase/uppercase charset; **Uppercase** — restores the default uppercase/graphics charset |

**Syntax:**

```
.charset lower    ; switch to lowercase charset
.charset upper    ; switch back to uppercase/graphics charset
```

**Generated code:**

Lowercase mode:

```
    LDA $D018
    ORA #$02      ; set bit 1 → lowercase/uppercase ROM at $1800
```

```
    STA $D018
```

Uppercase mode:

```
    LDA $D018
    AND #$FD      ; clear bit 1 → uppercase/graphics ROM at $1000
    STA $D018
```

**Size:** 8 bytes (LDA abs + ORA/AND imm + STA abs).

**Why ORA/AND instead of a direct write?** `$D018` also controls screen RAM location (bits 7–4). Toggling only bit 1 preserves the rest of the register.

**Typical workflow:**

```
.charset lower
.text 0, 0, "hello world", lower
.charset upper
```

> **Note:** The CHARSET macro only changes the VIC character ROM pointer. It does not call `$E544` (KERNAL charset init). For most cases this is sufficient; call `JSR $E544` first only if you need the KERNAL's own print routines to respect the change.

---

## CHARDEF

Defines a single 8×8 custom character in a RAM-based character set. Emits inline runtime code that copies 8 bytes into `base + index * 8` at execution time — no need for a preceding label or `ORG`.

| Argument | Description |
|---|---|
| Charset base | Base address of the RAM charset (default `$3800`). Must be aligned so the VIC-II can see it (see below). |
| Character index | Which character slot to redefine, 0–255. `65` = 'A' in the default screen code layout. |
| 8 bytes | Comma-separated bitmap rows, top-to-bottom. Each byte's bit 7 = leftmost pixel. |

**Syntax:**

```
.chardef $3800, 65, $18,$3C,$66,$7E,$66,$66,$66,$00
```

**Generated code (8 × `LDA #b` / `STA target+n`, 40 bytes total):**

```
LDA #$18
STA $3A08
LDA #$3C
STA $3A09
LDA #$66
STA $3A0A
LDA #$7E
STA $3A0B
LDA #$66
STA $3A0C
LDA #$66
STA $3A0D
LDA #$66
STA $3A0E
LDA #$00
STA $3A0F
```

**Target address:** `$3800 + 65 * 8 = $3A08`. Computed at compile time and hard-coded into the STA operands.

**Size:** 40 bytes per character (8 × 5).

**Typical workflow:**

```
; Point VIC-II at RAM charset at $3800
LDA $D018
AND #$F1
ORA #$0E       ; bits 3-1 = %111 → charset at bank+$3800
STA $D018

.chardef $3800, 65, $18,$3C,$66,$7E,$66,$66,$66,$00   ; redefine 'A'
.chardef $3800, 66, $7C,$66,$66,$7C,$66,$66,$7C,$00   ; redefine 'B'
; ... more .chardef lines for each custom char
```

**When to use CHARDEF vs alternatives:**

| Approach | Use when |
|---|---|
| **CHARDEF** | You need a few custom characters (say 1–20). Runtime cost is 40 bytes each. |
| **RAWBYTES @ $3800** | You have a full custom charset (256 chars). Total 2 KB of data, no runtime copy. |
| **INCBIN "charset.bin" @ $3800** | External charset file (built by the Character Editor). Cleanest option. |

| Approach | Use when |
|---|---|
| **Charset Canvas + INCBIN** | Full 256-character bitmap painted as one 128×128 image. |

> **Alignment reminder:** VIC-II expects the charset base at a multiple of `$0800`. Valid banks: `$0000`, `$0800`, `$1000`, … , `$3800` (within the current 16 KB VIC bank). RAM charsets typically live at `$2000`, `$2800`, `$3000` or `$3800`.

---

BOX_HIT

Axis-aligned bounding-box (AABB) collision test between two rectangles described by 4-byte zero-page structures. Returns the result in the accumulator: **A = 1** on overlap, **A = 0** otherwise. Pure inline assembly, no subroutine call.

| Argument | Description |
|---|---|
| Box1 ZP address | Zero-page base of the first box's 4-byte struct (default `$FB`). |
| Box2 ZP address | Zero-page base of the second box's 4-byte struct (default `$F7`). |

**Struct layout** (4 bytes per box, unsigned 8-bit coordinates):

| Offset | Field |
|---|---|
| `+0` | Left |
| `+1` | Top |
| `+2` | Right |
| `+3` | Bottom |

**Syntax:**

```
.box_hit $FB, $F7
```

**Generated code (30 bytes, fully PC-relative — no subroutine, no absolute jumps):**

```
    LDA $FD        ; b1 right
    CMP $F7        ; b2 left
    BCC no         ; R1 < L2 → miss
    LDA $F9        ; b2 right
    CMP $FB        ; b1 left
    BCC no
    LDA $FE        ; b1 bottom
    CMP $F8        ; b2 top
    BCC no
    LDA $FA        ; b2 bottom
    CMP $FC        ; b1 top
```

```
    BCC no
    LDA #$01       ; hit
    BNE done       ; unconditional (A ≠ 0)
no: LDA #$00
done:
```

**Size:** 30 bytes.

**Typical workflow:**

```
    ; Player sprite box at $FB..$FE
    LDA sprite_x            ; L
    STA $FB
    LDA sprite_y            ; T
    STA $FC
    CLC
    ADC #23                 ; +23 → B (24-pixel tall sprite)
    STA $FE
    LDA $FB
    CLC
    ADC #23                 ; R
    STA $FD

    ; Enemy box at $F7..$FA (populated similarly)
    ; ...

    .box_hit $FB, $F7        ; test player vs enemy → A = 0 or 1

    CMP #$01
    BNE no_collision
    JSR handle_hit
no_collision:
```

**Constraints:**

- Both zero-page addresses must be `≤ $FC` (each box needs 4 consecutive bytes: `zp`, `zp+1`, `zp+2`, `zp+3`).
- Coordinates are treated as **unsigned 8-bit** (0–255). For signed sprite coordinates outside this range, normalize before storing.
- The two boxes may overlap in ZP space if desired, but usually you want 8 distinct bytes.

**Why not a subroutine?** Inline generation avoids the JSR/RTS overhead (14+ cycles) and keeps the test hot in cache for tight game loops. If you need to test many pairs, put your own `JSR box_hit_sub` around a single BOX_HIT line manually.

**Comparison with UB's `box_hit()` function:** Ultimate Basic wraps the same 6502 logic as a runtime function returning to a variable. In VA, you place BOX_HIT inline where you need the test; the result is in `A`.

---

INCBIN

Like **BLOAD in BASIC** — picks up an external binary file ( `.bin` , `.prg` , `.sid` , `.raw` ) and embeds it directly into the assembled PRG at the address you specify.

| Argument | Description |
|---|---|
| File | Browse to select a `.bin` , `.prg` , `.sid` , or `.raw` file |
| Address | Target load address (e.g. `$C000` ) |

**Syntax:**

```
.incbin "music.bin", $C000
```

**Generated code comment:**

```
; INCBIN "music.bin" @ $C000 (2048 bytes)
.byte $01, $02, ...
```

**Size in code:** 0 bytes (deferred data section). The binary is embedded at the given address.

---

KOALA

Loads a **Koala Painter** picture ( `.koa` / `.kla` , 160×200 multicolor) and shows it. The picture data is embedded at a fixed address like `INCBIN` (the file's 2-byte `$6000` load address is stripped), and the directive itself compiles to an 86-byte display routine that runs where it sits.

| Argument | Description |
|---|---|
| File | Browse to select a `.koa` / `.kla` file (10003 bytes, or 10001 without load address) |
| Address | Bitmap address: `$6000` (default, Koala native, VIC bank 1, screen RAM `$4400` ) or `$2000` (VIC bank 0, screen RAM `$0400` ) |

The display routine copies the screen RAM part to the screen and the color RAM part to `$D800` , writes the background byte to `$D021` , selects the VIC bank in `$DD00` (keeping the other port bits) and switches on multicolor bitmap mode ( `$D011 = $3B` , `$D016 = $18` , `$D018 = $18` ). Other addresses are rejected: the remaining 8K slots are under the BASIC/KERNAL ROM (the copy loop would read ROM) or overlap the character-ROM shadow.

**Syntax:**

```
.koala "picture.koa"
.koala "picture.koa", $2000
```

**Example — show a picture and wait:**

```
* = $0801
.koala "picture.koa"
wait:
    JMP wait
```

**Size in code:** 86 bytes (display routine); the 10001 bytes of picture data go to the given address. The Hires editor of C64 Visual Assembler exports a compatible file.

---

LEVEL / LEVEL_ROW

Puts a **Level Editor** project into your program and copies level rows to a screen buffer — the building blocks of a scrolling game. The `level-demo.asm` sample of C64 Visual Assembler is a complete joystick-scrolled level.

**LEVEL**

An INCBIN-style directive that emits **0 bytes of code**. Only the **placed** screens of the Level Editor project (the `.bin` the Level Editor of C64 Visual Assembler saves) go into memory from the given address (default `$2000`), in slot order, 2000 bytes per screen (1000 screen codes + 1000 colors). Identical screens are stored once, and all empty slots share one blank screen. The pointer tables follow the screens. A level can be up to 7 × 10 screens.

**Syntax:**

```
.level "level.bin"
.level "level.bin", $4000
```

| Defined | Meaning |
|---|---|
| `LEVEL_SLOT_LO` / `LEVEL_SLOT_HI` | Address of the screen in each slot (row-major, `LEVEL_W` slots per row) |
| `LEVEL_ROW_SLOT` | Level character row → first slot of its screen row |
| `LEVEL_ROW_LO/HI`, `LEVEL_CROW_LO/HI` | Level character row → offset of its screen-code row / color row inside a screen |

| Defined | Meaning |
|---|---|
| `LEVEL_COL_SLOT`, `LEVEL_COL_OFS` | View start column → screen column and column inside the screen |
| `LEVEL_W`, `LEVEL_H` | Level size in screens |
| `LEVEL_COLS`, `LEVEL_ROWS` | Level size in characters |
| `LEVEL_MAX_X`, `LEVEL_MAX_Y` | Largest pixel scroll position of a 40×25 view |
| `LEVEL_BG`, `LEVEL_MC1`, `LEVEL_MC2`, `LEVEL_MULTICOLOR` | Background / multicolor colors and mode of the level |
| `LEVEL_DATA`, `LEVEL_SIZE`, `LEVEL_END` | Start, size and first free address after the level data |
| `LEVEL_OBJ_X_LO/HI`, `LEVEL_OBJ_Y_LO/HI`, `LEVEL_OBJ_TYPE`, `LEVEL_OBJ_PARAM` | Object table (Level Editor object layer): position in level pixels, type and parameter byte, in scroll order — `LEVEL_OBJ_COUNT` entries (max 255) |
| `LEVEL_OBJ_T_START` … `LEVEL_OBJ_T_TRIGGER` | Object type numbers: START 0, ENEMY 1, ITEM 2, DOOR 3, EXIT 4, TRIGGER 5 |
| `LEVEL_START_X`, `LEVEL_START_Y` | Position of the first Start object (only defined when the level has one) |
| `LEVEL_ATTR`, `LEVEL_ATTR_SOLID` … `LEVEL_ATTR_COLLECT` | Character attribute table (one flag byte per screen code; only when the level has attributes) and its bits: 1, 2, 4, 8 |
| `LEVEL_BORDER`, `LEVEL_SCR_COLORS`, `LEVEL_SCR_BG/BORDER/MC1/MC2` | Level border; per-slot colour tables (row-major like `LEVEL_SLOT_*`) when a screen has its own colours (`LEVEL_SCR_COLORS` = 1) |

Typical guard: `.assert LEVEL_END <= $9200, "level data runs into the code"`.

**LEVEL_ROW**

Copies one 40-character row of the level to a buffer: the level row in **X**, starting at the level column held in the `col` variable, to the address held in the `dst` pointer (`dst` / `dst+1`, lo/hi). With the third argument `color` it copies the colors instead of the screen codes (for color RAM). A 40-character window that spans two screens is handled with a self-modifying copy loop (~14 cycles per byte). X is kept, A and Y are destroyed. Needs a `.level` block in the program.

**Syntax:**

```
.level_row dst_lo, view_x
.level_row dst_lo, view_x, color
```

**Size in code:** 132 bytes — use `LBEQ` / `LBNE` when a branch has to jump over it.

---

SID

Like **BLOAD for music** — loads a `.sid` file into your PRG and automatically reads its Init and Play addresses from the header. Call Init once at startup, then call Play from your IRQ handler every frame.

| Argument | Description |
|---|---|
| File | The `.sid` file (PSID / RSID) |
| Custom address (optional) | Override the SID's native load address (e.g. `$1000`). Leave empty to use the address from the SID header. |

The SID header gives:

- **Load address** — where the data is placed in memory (the custom address when one is given)
- **Init address** — call this with JSR to initialize the music (moved together with a custom address)
- **Play address** — call this with JSR on every frame in an IRQ handler (moved together with a custom address)

**Syntax:**

```
.sid "Ikari_Warriors.sid"
.sid "Ikari_Warriors.sid", $1000
```

**Generated code comment:**

```
; SID "Ikari_Warriors.sid" @ $1000  Init:$1000  Play:$1006  (4096 bytes)
```

**Size in code:** 0 bytes inline. The SID binary is placed at the specified address as a deferred chunk in the PRG.

> **Important:** Most SID files contain hardcoded internal absolute addresses. They can only be relocated if the entire binary is shifted by the same offset. If a SID has internal jumps to `$10xx`, it must remain at `$1000` — moving it to a different address will break those internal references.

> **Typical usage:** Give the SID an address of its own (`.sid "tune.sid", $1000`) when its native one clashes with your code. Call Init once at startup, then call Play every frame from a raster IRQ handler — or let SID_PLAYER do both.

**Labels:** the SID directive defines `SID_LOAD` (where the data is placed), `SID_INIT` and `SID_PLAY` (the init and play addresses, moved together with a custom address). `SID_PLAY` is missing when the tune has no play address (an RSID that installs its own IRQ). Use them in your own code, e.g. `JSR SID_PLAY` in an IRQ handler.

---

SID_PLAYER

Plays the tune of the **SID** directive with one line: it initialises the tune and keeps calling its play routine from a raster interrupt, so your program can go on with its own work.

| Argument | Description |
|---|---|
| Song | Subtune number passed to the init routine in A (default `0`) |

| Argument | Description |
|---|---|
| Raster | Raster line of the interrupt, `$00` – `$FF` (default `$FA`) |

What it does: `SEI`, `LDA #song` / `JSR SID_INIT`, switches the CIA1 timer IRQ off (`$DC0D`), sets the IRQ vector (`$0314/$0315`) to its built-in handler, the raster line (`$D012`, bit 8 of `$D011` cleared) and the raster IRQ (`$D01A`), `CLI`, then jumps over the handler. The handler acknowledges the interrupt (`ASL $D019`), calls `JSR SID_PLAY` and goes on to the KERNAL (`JMP $EA31`, which also scans the keyboard).

**Syntax:**

```
.sid "Ikari_Warriors.sid"
.sid_player
.sid_player 1, $80
```

**Size in code:** 58 bytes.

> **Note:** it needs a `.sid` line in the program (`SID_INIT` and `SID_PLAY`); otherwise the build stops with an error. With the CIA1 timer IRQ off, the KERNAL runs once a frame from the raster IRQ — keyboard and `GETIN` keep working.

---

INCLUDE

Like **MERGE in BASIC** — pulls in another file and expands it inline at this position. Perfect for reusable subroutine libraries.

Two file kinds are supported:

- **Plain assembly source** (`.inc`, `.asm`, `.s`) — the file is read as VMA source. It is read again at every build, so you can edit it with any editor.
- **Visual Assembler project** (`.json`) — the blocks of a C64 Visual Assembler block program.

| Argument | Description |
|---|---|
| File | A `.inc` / `.asm` / `.s` source or a `.json` Visual Assembler project |
| Load address (optional) | If set (hex, e.g. `C000`), the included code is placed at that address — an `ORG` is put in front of it, and the included file's own ORGs are ignored. Without it the included file's own ORGs control placement. |

**Syntax:**

```
.include "library.json"
.include "macros.inc", $1500
.include "sprites.asm"
```

- Give the **file extension** — a bare name like `.include "macros"` is treated as `.include "macros.json"`.
- Path resolution: next to the source first, then in every search folder (the CLI's `--base-dir` / `--search`); a path written on another machine falls back to its bare file name. A missing include is not an error — the build goes on without it.

**Generated code (no address override):**

```
; .include "library.json"
... (the included code follows)
```

**Generated code (with load address `C000`):**

```
; .include "library.json" @ $C000
*=$C000
... (the included code follows)
```

> **Tip:** Use INCLUDE to build reusable subroutine libraries that you can share across projects. `.inc` / `.asm` / `.s` files are best when you want to edit the library in a plain text editor or share it with other 6502 assemblers; `.json` when the library is a Visual Assembler block program. Set a load address when the library has no ORG of its own, or when you want to override its default placement.

> **2.1:** an include that is not found is an error, and so is a file that includes itself (directly or further down). An included file's own `.include`, `.incbin` and `.sid` lines are loaded too, relative to its own folder, and its macro calls are expanded — up to 16 levels deep.

---

TABLE

Like **DIM at a specific address** — names a lookup table and sets where it lives in memory. Put `.byte`, `.word` or `.fill` lines after it to define the table's contents.

| Argument | Description |
|---|---|
| Name | Label identifier for the table (e.g. `color_table`) |
| Address | Fixed address where the table starts (e.g. `$C000`) |

**Syntax:**

```
.table color_table $C000
.table lookup              ; at the current address
```

**Generated code:**

```
color_table:
```

With an address the program counter jumps there; without one the table starts at the current address. Put `.byte` / `.word` / `.fill` lines after it to fill the content.

**Size:** 0 bytes.

---

ORG

Sets where in memory the program (or a section of it) is placed — like choosing a start address before typing in machine code. Every program needs at least one ORG. The standard C64 BASIC-loadable start is `$0801`.

| Argument | Description |
|---|---|
| Address | The new origin address (e.g. `0801` in HEX, or `2049` in DEC) |
| HEX / DEC | Toggle the address input between hexadecimal and decimal display |

**Syntax:**

```
* = $C000
```

**Generated code:**

```
* = $C000
```

**Size:** 0 bytes. ORG itself generates no machine code.

Each ORG starts a new section. The lines that follow are assembled starting at that address. The PRG holds all sections in one file — gaps between sections are filled with zeros.

**Example — code at `$0801`, data table at `$C000`:**

```
* = $0801
    LDX #$00
loop:
    LDA $C000,X
    STA $D800,X
    INX
    BNE loop
    RTS

* = $C000
    .byte $01, $02, $03, ...
```

> **Tip:** Without an ORG the program starts at `$0801` (2049 decimal), the start of BASIC. With the BASIC SYS stub (the default) the assembler puts a short `10 SYS…` line at `$0801` and your code starts at `$080D`. See Output: PRG, BASIC stub and entry point.

---

### LOOP / NEXT

Like `FOR X=N TO 1 STEP -1 : ... : NEXT X` in BASIC — counts down from N to 1 using the X or Y register. Put your instructions between LOOP and NEXT, and it loops the right number of times automatically.

**LOOP**

| Argument | Description |
|---|---|
| Register | `X` or `Y` — the counter register |
| Count | Loop iteration count (hex or decimal, e.g. `0A` = 10) |
| Label | Auto-generated loop label (e.g. `loop0`) |

**Syntax:**

```
.loop X, 10, loop0
```

**Generated code:**

```
    LDX #$0A
loop0:
```

**Size:** 2 bytes (LD_ opcode + immediate operand).

**NEXT**

| Argument | Description |
|---|---|
| Register | Automatically matched to the LOOP register |
| Label | Automatically linked to the LOOP label |

**Syntax:**

```
.next loop0
```

**Generated code:**

```
    DEX
    BNE loop0
```

**Size:** 3 bytes (DEX + BNE + branch offset).

**Example — clear 10 screen cells:**

```
    LDX #$0A
loop0:
    LDA #$20        ; space character
    STA $0400,X
    DEX
    BNE loop0
```

---

FOR / ENDF

Like `FOR X=0 TO N-1 : ... : NEXT X` in BASIC — counts *up* from 0. Ideal when you need a forward index, e.g. stepping through a string or an array.

**FOR**

| Argument | Description |
|---|---|
| Register | `X` or `Y` — the counter register |
| Count | Loop limit (hex or decimal, e.g. `$12` = 18). X/Y runs from 0 up to limit-1. |
| Label | Auto-generated loop label (e.g. `for0`) |

**Syntax:**

```
.for X, $12, for0
```

**Generated code:**

```
    LDX #$00
for0:
```

**Size:** 2 bytes (LD_ opcode + `#$00`).

**ENDF**

| Argument | Description |
|---|---|
| Register | Automatically matched to the FOR register |
| Label | Automatically linked to the FOR label |
| Count | Automatically copied from the paired FOR |

**Syntax:**

```
.endf for0
```

**Generated code:**

```
    INX
    CPX #$12
    BNE for0
```

**Size:** 5 bytes (IN_ + CP_ #imm + BNE offset).

**Example — print a null-terminated string:**

```
    LDX #$00
for0:
    LDA msg,X       ; msg = PETSCII string at fixed address
    BEQ done        ; null terminator → exit
    JSR $FFD2       ; CHROUT
    INX
    CPX #$12        ; 18 characters max
    BNE for0
done:
    RTS
```

> **LOOP vs FOR:** LOOP counts down (N→1) — good for delays, fills, pixel loops. FOR counts up (0→N) — good for string/array access. Both can use X or Y.

---

PUSH / PULL

Like **saving variables before a GOSUB and restoring them after** — but uses the 6502 hardware stack. If a subroutine uses A, X, or Y, wrap it with PUSH and PULL so the calling code's registers are preserved.

**PUSH**

Pushes one or more registers onto the stack. The order is always A → X → Y (innermost first).

| Argument | Description |
|---|---|
| Registers | Any combination: `A`, `X`, `Y`, `AX`, `AY`, `XY`, `AXY` |

**Syntax:**

```
.push AXY
```

**Generated code (example: `AX`):**

```
    PHA
    TXA
    PHA
```

**Size:** 1 byte for A ( `PHA` ), 2 bytes for X or Y (transfer + push).

**PULL**

Restores registers from the stack in **reverse order** (Y → X → A).

| Argument | Description |
|---|---|
| Registers | Same as PUSH — must match the corresponding PUSH |

**Syntax:**

```
.pull AXY
```

**Generated code (example: `AX`):**

```
PLA
TAX
PLA
```

> **Rule:** PUSH and PULL must always use the **same register set**. `PUSH AX` → `PULL AX` (internally restores in reverse: X first, then A).

---

END / RTS alias

Like **RTS with a friendlier macro name** — `.end` emits a single `RTS` byte, a short subroutine terminator.

**Syntax:**

```
.end
```

**Generated code:**

```
RTS
```

```

```

**Size:** 1 byte.

Use this when you want an end-of-subroutine marker that reads a little more like a macro than a raw instruction.

---

MACRO / ENDM / INVOKE

Like **a named GOSUB with parameters** — define a reusable chunk of code once (MACRO…ENDM), then call it anywhere with INVOKE. Pass different argument values each time instead of copy-pasting code.

**MACRO (definition start)**

| Argument | Description |
|---|---|
| Name | Identifier for the macro (e.g. `setColor`) |
| Params | Optional comma-separated parameter names (e.g. `color` or `color, count`) |

Marks the start of a macro definition. The lines between MACRO and ENDM are the macro's body. A macro with parameters **generates no code** where the definition sits; a macro without parameters is also assembled there as a subroutine (`JSR name` calls it). In the body a parameter is written as `{paramName}` or as its bare name; the call fills in the argument — in instruction operands and in the arguments of the library macros alike (`.set_border c`, `.sprite_pos n, x, y` …). Labels, LOOP labels and runtime `.if` / `.while` labels of the body are unique per call. A macro call inside a macro body is not expanded (an error).

**Generated code:**

```

; .MACRO setColor (color)
    ... (body)
; .ENDM

```

**Syntax:**

```

.macro setColor color
    LDA {color}
    STA $D020
.endm

```

**ENDM (definition end)**

Closes the current macro definition. No arguments.

**INVOKE**

Calls a defined macro at this position and substitutes the supplied argument values for `{paramName}` placeholders in the body.

| Argument | Description |
|---|---|
| Macro name | A macro defined in the program or in an included file |
| Arguments | Comma-separated argument values matching the macro's parameter list (e.g. `#$07`) |

**Generated code:**

```
; .invoke setColor(#$07)
    LDA #$07
    STA $D020
```

**Syntax:**

```
; single argument:
.invoke setColor(#$07)

; multiple arguments:
.invoke drawPixel($10, $20)

; no arguments:
.invoke clearScreen

; text / string arguments (quoted):
.invoke printText("Hello, World!")

; .call alias (synonym for .invoke):
.call setColor(#$07)
```

The macro body is expanded inline with `{paramName}` replaced by the actual arguments. The space-separated form (`.invoke setColor #$07`) is also accepted.

**Argument types:**

- **Numeric**: `#$07`, `$10`, `255` — hex or decimal values
- **Text strings**: `"Hello, World!"` — quoted strings (the quotes are removed); commas inside quotes are treated as part of the text, not as argument separators
- **Character literal**: `'A'` — stays a character literal, so the body gets its code (`$41`)
- **Mixed**: `#$07, "hello", $20` — any combination

> **Tip:** Define macros at the top (or bottom) of your program, then INVOKE them wherever needed. Macros can be invoked multiple times with different arguments.

> **2.1:** a macro may call another macro ( `both(a, b)` calling `setc(a)` ); the inner call gets the outer call's arguments and its own unique local labels. A macro that calls itself stops with an error after 16 levels instead of hanging.

---

### REGION / ENDREGION

Purely a grouping for editors and listings — **zero bytes**, zero effect on the assembled code.

| Argument | Description |
|---|---|
| Region name | Free-text label for the section (e.g. `init` , `game_loop` , `sprite_setup` ) |

**Syntax:**

```
.region init
    ; code...
.endregion
```

**Generated code:**

```
; region init
    SEI
    LDA #$00
    STA $D020
; endregion init
```

**Size:** 0 bytes for both REGION and ENDREGION.

> **Note:** Regions can be **nested** inside each other. Each ENDREGION closes the nearest open REGION. No effect on the assembled output.

---

### DEFINE / IF / ELSE / ENDIF

Like **a switch the assembler reads** — `DEFINE DEBUG` turns on a symbol, then the lines of any `IF DEBUG` are assembled and its `ELSE` branch is skipped. Remove the DEFINE and the IF part disappears from the output. No need to delete code for release builds.

**DEFINE**

| Argument | Description |
| --- | --- |
| Symbol | One or more comma-separated identifiers to activate (e.g. `DEBUG` or `DEBUG, PAL`) |

**Syntax:**

```
.define DEBUG, PAL
```

**Generated code:**

```
; .DEFINE DEBUG
; .DEFINE DEBUG, PAL
```

A `DEFINE` can activate multiple symbols at once (comma-separated). Put DEFINEs at the top of your program.

**IF**

| Argument | Description |
| --- | --- |
| Condition | A `.define`d symbol (active), a `.const` name (active when not zero) or a constant expression (`LEVEL == 2`, `MODE & 1`) |

**Syntax:**

```
.if DEBUG
```

**Generated code:**

```
; .IF DEBUG
```

The lines between `IF` and `ENDIF` (or `ELSE`) are assembled or skipped by the condition. A bare name is true when it is `.define`d or a non-zero `.const`, false when it is neither; any other condition must be a constant expression — one that cannot be evaluated at assembly time is an error. Skipped lines appear as `; [IF skipped] …` comments and generate **zero bytes**.

**ELSE**

No arguments. Marks the alternative branch — assembled when the `IF` condition is *not* active.

**Syntax:**

```
.else
```

**Generated code:**

```
; .ELSE
```

**ENDIF**

No arguments. Closes the conditional part.

**Syntax:**

```
.endif
```

**Generated code:**

```
; .ENDIF
```

**Size:** 0 bytes for all four directives. Only the content *between* them counts.

**Example — debug border flash, release build skips it:**

```
; .DEFINE DEBUG

; .IF DEBUG
    LDA #$02        ; red border
    STA $D020
; .ELSE
    LDA #$00        ; black (release)
```

```
    STA $D020
; .ENDIF
```

**Example — multiple symbols in one DEFINE:**

```
; .DEFINE DEBUG, PAL

; .IF PAL
    LDA #$xx        ; PAL timing constant
; .ELSE
    LDA #$xx        ; NTSC timing constant
; .ENDIF
```

Nested `IF`s are supported. If an outer part is skipped, the inner ones are skipped too.

> **Note:** This is compile-time condition handling. For runtime compare/branch sugar, see **Runtime IF / ELSE / ENDIF** below.

.ASSERT

A **compile-time sanity check**. `.assert` evaluates an expression while assembling; if it is false (`0`), the build stops with a clear error that includes the actual value. If it is true (non-zero), it emits nothing.

| Argument | Description |
|---|---|
| Expression | Any assembler expression: labels, `CONST`s, `*` (program counter), arithmetic, and comparisons (`<`, `<=`, `>`, `>=`, `==`, `!=`) |
| Message | Optional text appended to the failure error |

**Syntax:**

```
.assert spriteData < $C000
.assert * < $A000
.assert end - start <= 256, "sprite table overflowed one page"
```

**Behaviour:**

- **Size:** 0 bytes.
- A false assertion aborts assembly:
  `` `.assert end - start <= 256` is false (value: 0). sprite table overflowed one page ``
- An assertion that cannot be evaluated (undefined label, etc.) also fails, with
  "*cannot be evaluated at assembly time*".

- Comparisons yield `1` / `0`; place `.assert` anywhere in the program flow — it is checked at the address it sits on, so `.assert * < $D000` tests the current output position.

**Example — guard sprite data against a page crossing:**

```
* = $3000
sprite_data:
    .byte $00, $00, $00        ; … 63 bytes …
sprite_data_end:
    .assert sprite_data_end - sprite_data == 63, "sprite must be exactly 63 bytes"
    .assert (sprite_data % 64) == 0, "sprite must be 64-byte aligned"
```

---

## CONST

Like **a named variable that never changes** — `SCREEN = $0400`. Use the name instead of typing raw addresses everywhere, making the code easier to read and change later.

| Argument | Description |
|---|---|
| Name | Identifier for the constant (e.g. `SCREEN`) |
| Value | Numeric value in the selected base (e.g. `0400` in HEX = address $0400), or a PC-relative expression (see below) |
| Format | HEX or DEC — controls how the value is entered and displayed |

**Syntax:**

```
.const SCREEN = $0400
.const FRAMES_1S = 60
```

**Generated code:**

```
; .CONST SCREEN = $0400
```

**PC-relative expressions ( `*+N` / `*-N` ):**

The value also accepts `*+N` or `*-N`, where `*` is the address of the `.const` line itself. This is used to create a named alias for a byte inside a nearby instruction — the classic self-modifying code pattern:

```
.const op = *+1      ; op → address of the immediate operand of the next LDA
LDA #$00            ; $00 will be patched at runtime
...
STA op              ; overwrites the #$00 byte → LDA reads the new value next time
```

The CONST emits 0 bytes; the label resolves at compile time to `current_address + 1`.

**Arithmetic expressions:**

The value accepts general arithmetic, including references to previously defined CONST names, hex/binary literals, and built-in math functions:

```
.const SCREEN      = $0400
.const SCREEN_END  = SCREEN + 40*25   ; 1000 bytes later
.const COLOR_RAM   = $D800
.const MID_X       = 160
.const SIN_TABLE   = round(sin(PI/8) * 127)   ; pre-computed sine value
```

**Built-in functions:** `sin()`, `cos()`, `round()`, `max(a,b)`, `min(a,b)`, `abs()`, constant `PI`

Operators: `+ - * /` Literals: `$FF` (hex), `%10110000` (binary) Low/high byte: `lo(expr)`, `hi(expr)`

**Size:** 0 bytes.

---

VAR

Like **CONST, but auto-allocated** — `VAR` reserves zero-page storage for a label without you having to type the address. Use it for counters, pointers, and short-lived state that belongs in ZP.

| Argument | Description |
|---|---|
| Name | Variable name / label |
| Size (optional) | Number of bytes to reserve. Omit for a single byte. |

**Syntax:**

```
.var counter
.var timer, 2
.var lives
```

**Practical example:**

```
.region Vars
.var counter
.var timer, 2
.endregion

LDA #$00
STA counter
```

**Generated code:**

```
; .var counter
```

**Size:** 1 byte by default, or `N` bytes when size is specified.

The allocator walks a configurable zero-page cursor ( `$02` to `$FE` ) and assigns the next free slot. If the requested region overlaps an already-used label, the compiler emits a warning.

---

Runtime IF / ELSE / ENDIF

Like **a real branch template** — this version works at runtime, not compile time. It compares `A` , `X` , or `Y` against an immediate value and emits the correct `CMP` / `CPX` / `CPY` + branch sequence for you.

| Argument | Description |
| --- | --- |
| Register | `A` , `X` , or `Y` |
| Operator | `==` , `!=` , `<` , `<=` , `>` , `>=` |
| Value | Immediate value in HEX or DEC format |

**Syntax:**

```
.if A == #$10
    LDA #$07
.else
    LDA #$0F
.endif
```

**Size:** Depends on the chosen branches and compare form.

The comparison is unsigned by default. For `<=` and `>` the macro expands to the shortest equivalent branch chain for the selected register.

---

### WHILE / ENDW

Like **a runtime loop with a test at the top** — the body runs while the condition stays true.

| Argument | Description |
|---|---|
| Register | `A`, `X`, or `Y` |
| Operator | `==`, `!=`, `<`, `<=`, `>`, `>=` |
| Value | Immediate value in HEX or DEC format |

**Syntax:**

```
.while A != #$00
    JSR getchar
.endw
```

**Size:** Depends on the loop body and compare form.

Use `WHILE` when the loop may end before the first iteration finishes. It is the runtime counterpart of the count-based `LOOP / NEXT` helper.

---

### REPEAT / UNTIL

Like **a runtime loop with a test at the bottom** — the body always runs at least once, then the condition decides whether to stop.

| Argument | Description |
|---|---|
| Register | `A`, `X`, or `Y` |
| Operator | `==`, `!=`, `<`, `<=`, `>`, `>=` |
| Value | Immediate value in HEX or DEC format |

**Syntax:**

```
.repeat
    JSR getchar
```

```
.until A == #$00
```

**Size:** Depends on the loop body and compare form.

Use `REPEAT / UNTIL` when you want the body to execute at least once before the exit check.

---

### MEMCPY / MEMSET

Like **small memory routines you reach for constantly** — `MEMCPY` copies a contiguous block, `MEMSET` fills a range with one byte.

| Macro | Arguments |
|---|---|
| `MEMCPY` | `src`, `dst`, `size` |
| `MEMSET` | `addr`, `value`, `size` |

**Syntax:**

```
.memcpy src=$C000, dst=$D000, size=$0100
.memset addr=$0400, value=#$20, size=$03E8
```

**Generated code:** inline copy/fill loops, chosen to match the requested size.

Sizes up to 256 bytes use a short 8-bit loop. Larger sizes switch to a 16-bit counter automatically.

> **2.1:** the size must be known when the code is laid out — a number or a constant. A label defined later (whose value the layout does not know yet) is an error instead of code that does not fit its place.

---

### PRINT / PRINT_CHAR / PRINT_HEX / CLEAR_SCREEN / WAIT_KEY / DELAY / SET_BORDER / SET_BG

**PRINT**

Like **PETSCII output without the boilerplate** — prints a string through `CHROUT` with the same uppercase/lowercase handling (`lower`) as `.petscii`.

**Syntax:**

```
.print "HELLO"
.print "hello", lower
```

**PRINT_CHAR**

Prints one PETSCII byte by numeric code and sends it through `CHROUT`. The value can also be a named const or label that resolves to a byte at assemble time.

**Syntax:**

```
.print_char 65
.print_char $41
.print_char color
```

**PRINT_HEX**

Prints an 8-bit value as hexadecimal text through the normal KERNAL output path.

**Syntax:**

```
.print_hex A
```

**CLEAR_SCREEN**

Shortcut for the standard C64 clear-screen control code.

**Syntax:**

```
.clear_screen
```

**WAIT_KEY**

Waits until a key is pressed, so you do not have to hand-roll the `GETIN` loop every time.

**Syntax:**

```
.wait_key
```

**DELAY**

Waits the requested number of frames through a shared helper routine. Use this for short pauses and timing gaps when a full custom loop would be overkill. The frame count can be a raw number or a named const, and `.wait` is just an alias of `.delay`.

**Syntax:**

```
.delay 29
.wait 29
.delay frames=FRAMES_1S
```

**SET_BORDER / SET_BG**

Convenience wrappers for the VIC-II color registers. The color value can be a raw number or a named const that resolves to 0–15.

**Syntax:**

```
.set_border 6
.set_bg 0
.set_border color
.set_bg color
```

**Size:** Each helper expands to a tiny register write sequence or a short KERNAL call.

---

IRQ_SETUP

Sets up a raster IRQ handler in one step. The macro writes the IRQ vector, enables raster IRQs, sets the line, disables the common CIA IRQ sources, and returns to normal execution with `CLI`.

| Argument | Description |
|---|---|
| Handler | IRQ routine label (e.g. `my_irq`) |
| Raster | Raster line in hex or decimal (e.g. `$FA`) |

**Syntax:**

```
.irq_setup handler=my_irq, raster=$FA
```

**Size:** A small setup sequence; exact length depends on the selected raster line.

Use this when you want the common "SEI / install handler / enable IRQ / CLI" boilerplate without scattering it through the program.

---

RAND

Like **a tiny built-in PRNG** — returns an 8-bit pseudo-random value from a compact zero-page seed.

| Argument | Description |
|---|---|
| Seed | Optional zero-page seed byte or label (default `$FB`) |

**Syntax:**

```
.rand
```

**Size:** A handful of bytes, depending on the chosen implementation path.

The generator is meant for gameplay, effect variation, and quick test data. It is deliberately small rather than cryptographically fancy.

---

SPRITE_INIT

Sets up a VIC-II sprite in one line — instead of writing ~6 POKE statements in BASIC. Sets the sprite's data pointer, turns it on, optionally enables multicolor mode, and sets its colour.

| Argument | Description |
|---|---|
| Sprite # | Sprite number 0–7 |
| Colour | Colour index 0–15 (C64 palette) |
| Data page | Sprite data address / 64 (e.g. `$21` if data is at `$0840`) |
| Multicolor | Toggles the sprite's multicolor bit (`$D01C`) |

**Syntax:**

```
.sprite_init 0, 7, $21
.sprite_init 0, 7, $21, multicolor
.sprite_init 0, 7, $21, mono
```

**Generated code:**

```
    LDA #$21
    STA $07F8       ; sprite pointer register ($07F8 + N)
    LDA $D015
    ORA #$01        ; set enable bit for sprite 0
    STA $D015
    LDA $D01C
    AND #$FE        ; clear multicolor bit for sprite 0
    STA $D01C
    LDA #$07
    STA $D027       ; sprite 0 colour register
```

**Size:** 26 bytes.

> **Sprite data page:** `data_address ÷ 64`. With the default BASIC SYS stub, `ALIGN 64` after `JMP main` places sprite data at `$0840` → page = `$21`.

---

SPRITE_POS

Like `POKE 53248, x : POKE 53249, y` in BASIC — sets a sprite's starting position. The coordinates are baked in at assemble time and accept consts; for animation use `INC` / `DEC` on the sprite register directly.

| Argument | Description |
|---|---|
| Sprite # | Sprite number 0–7 |
| X | Horizontal position 0–319 |
| Y | Vertical position 0–255 |

**Syntax:**

```
.sprite_pos 0, 152, 100
```

**Generated code (example: sprite 0, X=152, Y=100):**

```
    LDA #$98        ; X low byte
    STA $D000       ; sprite 0 X register
    LDA $D010
    AND #$FE        ; clear X MSB for sprite 0 (X ≤ 255)
    STA $D010
```

```
    LDA #$64        ; Y = 100
    STA $D001       ; sprite 0 Y register
```

For X > 255 the macro sets the corresponding bit in `$D010` instead of clearing it.

**Size:** 18 bytes.

> **Note:** `SPRITE_POS` bakes the X/Y into the code ( `LDA #$xx` ). To animate a sprite at runtime use `INC $D000` / `DEC $D000` — see the `sprite-macro-demo` sample.

---

### WAIT_RASTER

Waits for the VIC-II electron beam to reach a specific scan line — like syncing to a TV frame. Put this at the top of your game loop to prevent sprite tearing. No JSR, no label needed.

| Argument | Description |
|---|---|
| Raster line | Target raster line in hex (e.g. `FF` = line 255) |

**Syntax:**

```
.wait_raster $FF
```

**Generated code:**

```
wait:
    LDA $D012       ; current raster line
    CMP #$FF        ; target line
    BNE wait        ; loop back (-7 bytes)
```

**Size:** 7 bytes (the `BNE` offset `$F9` = −7 always points back to the `LDA` ).

> **Tip:** Place `WAIT_RASTER` at the top of your game loop to synchronise with the display and prevent sprite tearing.

---

### JOYSTICK

Like reading **`PEEK($DC00)`** and then POKEing the sprite position — but in one line. Reads one CIA joystick port and adjusts a sprite's X/Y registers accordingly. Entirely inline, no JSR needed.

| Argument | Description |
|---|---|
| Port | `1` = port 1 ( `$DC01` ) or `2` = port 2 ( `$DC00` ) |
| Sprite # | Sprite number 0–7 (controls which X/Y register pair is updated) |

**Syntax:**

```
.joystick 2, 0
```

**Generated code (port 2, sprite 0):**

```
    LDA $DC00       ; read CIA port 2
    LSR             ; bit 0 → carry (Up)
    BCS skip_up     ; carry set = NOT pressed
    DEC $D001       ; Y−1 (move up)
skip_up:
    LSR             ; bit 1 → carry (Down)
    BCS skip_down
    INC $D001       ; Y+1 (move down)
skip_down:
    LSR             ; bit 2 → carry (Left)
    BCS skip_left
    DEC $D000       ; X−1 (move left)
skip_left:
    LSR             ; bit 3 → carry (Right)
    BCS skip_right
    INC $D000       ; X+1 (move right)
skip_right:
```

**Joystick bit map (active-LOW — bit = 0 means pressed):**

| Bit | Direction | CIA register |
|---|---|---|
| 0 | Up | $DC00 (port 2) / $DC01 (port 1) |
| 1 | Down | |
| 2 | Left | |
| 3 | Right | |
| 4 | Fire | (not handled by this macro) |

**Size:** 27 bytes. The `BCS` offset is always `+3` (skips the following 3-byte `DEC` / `INC abs` instruction).

**Variable mode — `.joystick port, vx, vy[, fire]`:** instead of moving a sprite, the direction is written into variables: `vx` / `vy` = `-1` / `0` / `+1` ( `$FF` / `$00` / `$01` ), and the optional `fire` = `1` while the button is pressed, `0`

otherwise. Use it for a scrolling camera, a player with its own physics, or menus. Port `3` reads both joystick ports at once ( `LDA $DC00` / `AND $DC01` ), so either port works.

```
.joystick 2, vx, vy          ; 35 bytes
.joystick 3, vx, vy, fire    ; both ports + fire: 45 bytes
```

**Typical usage:** Place inside a `gameloop` label with `WAIT_RASTER` first:

```
gameloop:
    WAIT_RASTER ($FF)
    JOYSTICK (port=2, sprite=0)
    JMP gameloop
```

---

**2.1:** the port must be known when the code is laid out (a number or a constant), because port 3 makes the code 3 bytes longer; a label defined later is an error.

MOUSE

Reads a Commodore 1351 proportional mouse and moves a sprite. Entirely **inline** — no JSR or label needed. The macro selects the CIA port, waits for the SID paddle inputs to settle, then decodes the delta movement using the standard 1351 driver pattern and applies it to the sprite registers.

| Argument | Description |
|---|---|
| Port | `1` = CIA `$DC00` bits `7:6` = `%01` ; `2` = `%10` |
| Sprite # | Sprite number 0–7 |
| ZP byte X | Zero-page address (hex) to hold the previous POTX sample (e.g. `FD` ) |
| ZP byte Y | Zero-page address (hex) to hold the previous POTY sample (e.g. `FE` ) |

**Generated code shape (port 1, sprite 0, ZP `$FD` / `$FE` ):**

```
    ; CIA port select + settle
    LDA $DC00
    AND #$3F
    ORA #$40
    STA $DC00
    LDX #$67
```

```
wait:
    DEX
    BNE wait

    ; X axis — standard 1351-style 7-bit delta decode
    LDA $D419
    TAY
    SEC
    SBC $FD
    AND #$7F
    LDX #$00
    CMP #$40
    BCS xneg
    LSR A
    BEQ xdone
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
    EOR #$01
    STA $D010
    JMP xdone
xneg:
    ORA #$C0
    CMP #$FF
    BEQ xdone
    SEC
    ROR
    DEX
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
    EOR #$01
    STA $D010
xdone:

    ; Y axis — same decode, then inverted before apply
    LDA $D41A
    TAX
    SEC
    SBC $FE
    AND #$7F
    CMP #$40
```

```
    BCS yneg
    LSR A
    BEQ ydone
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
    JMP ydone
yneg:
    ORA #$C0
    CMP #$FF
    BEQ ydone
    SEC
    ROR
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
ydone:
```

**Size:** 142 bytes.

**Syntax:**

```
.mouse port, spriteNum, zpX, zpY
; example:
.mouse 2, 0, FD, FE
```

**Important:** Before the first call, initialise the zero-page bytes with the current POTX/POTY values to avoid a jump on the first frame:

```
    ; port 1: LDA $DC00 : AND #$3F : ORA #$40 : STA $DC00
    ; port 2: LDA $DC00 : AND #$3F : ORA #$80 : STA $DC00
    LDA $D419 : LSR A : AND #$3F : STA $FD
    LDA $D41A : LSR A : AND #$3F : STA $FE
```

**Tip:** Poll the mouse once per frame — place `WAIT_RASTER` in the game loop before `MOUSE`.

---

SPRITE_COL

Like **`PEEK($D01E)`** in BASIC — checks the VIC-II hardware collision registers and tells you whether a sprite hit another sprite or the background. Entirely inline, no JSR needed.

| Argument | Description |
|---|---|
| Sprite # | Sprite number 0–7 (which sprite's bit to check) |
| Collision type | `Sprite-Sprite ($D01E)` — collision with another sprite; `Sprite-Background ($D01F)` — collision with background graphics |

**Syntax:**

```
.sprite_col 0, sprite
.sprite_col 0, background
```

**Generated code (sprite 0, sprite–sprite):**

```
    LDA $D01E       ; read sprite-sprite collision register (clears it!)
    AND #$01        ; isolate bit 0 (sprite 0)
                    ; A ≠ 0 → collision occurred
```

**Size:** 5 bytes.

> **Important:** Reading `$D01E` / `$D01F` **clears the register**. Read it once per frame and act on the result immediately with `BEQ` / `BNE`.

**Typical usage:**

```
gameloop:
    .wait_raster $FF
    .joystick 2, 0
    .sprite_col 0, sprite
    BEQ no_hit          ; A = 0 → no collision
    LDA #$02
    STA $D020           ; red border = hit!
    JMP gameloop
no_hit:
    LDA #$0E
    STA $D020           ; light blue border = clear
    JMP gameloop
```

> **See also:** `collision-demo` sample — green ball (sprite #0) vs. red cross (sprite #1).

---

LOADFILE

Like **`LOAD "file",8`** in BASIC — loads a file from a D64 disk at runtime using the KERNAL LOAD routine. Use this to load data, music, or extra code from disk while your program is running.

| Argument | Description |
|---|---|
| Filename | File name on the disk (max 16 chars, auto-uppercase; characters `,`, `"`, `/`, `\`, `:`, `*`, `?`, `<`, `>`, `\|` are filtered) |
| Device | Device number 8–30 (default `8`) |
| Override address (optional) | Hex load address (e.g. `C000`). If set, the file is loaded to this address (`sec=0`, ignoring the PRG header). Leave empty to use the file's own 2-byte PRG header (`sec=1`). |
| Error label (optional) | If set, a `BCS` instruction is generated after JSR LOAD. If the KERNAL returns with carry set (error), execution jumps to this label. |

**Syntax:**

```
.loadfile "DEMO-COLORS", 8
.loadfile "DEMO-COLORS", 8, $C000
.loadfile "DEMO-COLORS", 8, $C000, error_label
```

**Generated code structure:**

```
    JMP skip_filename      ; jump over the inline filename
    .byte "DEMO-COLORS"    ; filename bytes (PETSCII, 11 chars)
skip_filename:
    LDA #11                ; filename length
    LDX #<fname            ; pointer lo
    LDY #>fname            ; pointer hi
    JSR $FFBD              ; SETNAM
    LDA #1                 ; logical file #1
    LDX #8                 ; device 8
    LDY #0                 ; sec=0 (override addr) or #1 (file's own addr)
    JSR $FFBA              ; SETLFS
    LDA #0                 ; LOAD command (not VERIFY)
    JSR $FFD5              ; LOAD
    BCS fail               ; (only if error label set)
```

**Size:** `3 + filename_length + 9 (SETNAM) + 9 (SETLFS) + (4 if override) + 5 (LOAD) + (2 if error label)` bytes. Minimum 27 bytes.

> **Important:** The filename is stored inline in the machine code right after a `JMP skip_filename`. The file name on the disk must be uppercase PETSCII — which matches plain ASCII uppercase letters ( `A` – `Z` ). The macro enforces this automatically.

> **Always use an error label** for production programs — if the file isn't found, KERNAL sets the carry flag and execution falls through to whatever is next.

> **See also:** `loadfile-demo` sample — demonstrates loading `DEMO-COLORS.PRG` from a D64 with a BCS error branch and a visual error screen.

---

### EXODECRUNCH

In-program **Exomizer decompression**. Use this macro right after a `LOADFILE` that loaded an Exomizer `mem`-mode compressed stream — EXODECRUNCH unpacks it backward into the address embedded in the stream.

| Argument | Description |
|---|---|
| Depacker address | Where the depacker code lives in memory (default `B000`). Must be a 16-bit hex address. |

**Syntax:**

```
.exodecrunch
.exodecrunch depacker=$B000
```

**Generated code (19 bytes):**

```
    LDA $AE          ; KERNAL load-end lo (set by previous LOAD)
    STA $04          ; depacker src-end pointer lo
    LDA $AF          ; KERNAL load-end hi
    STA $05          ; depacker src-end pointer hi
    LDA #$36         ; BASIC ROM off ($A000-$BFFF becomes RAM)
    STA $01
    JSR depacker     ; depacker reads back-to-front via $04/$05
    LDA #$37         ; restore default mapping (BASIC + KERNAL + I/O)
    STA $01
```

**How it works:**

1. KERNAL `LOAD` ($FFD5) updates ZP `$AE/$AF` to point one past the last loaded byte. EXODECRUNCH copies this to ZP `$04/$05`, which is the official Exomizer backward source-end convention.
2. The depacker is typically placed at `$B000` (within the BASIC ROM mapped region). The macro toggles `$01 = $36` so the CPU sees RAM there during the JSR, then restores `$01 = $37` afterward.
3. The decompression target address is **encoded into the compressed stream itself** when you compress with `exomizer mem -l <load> file,<target>` — the depacker reads it from the stream's first bytes.

**Depacker binary:** the pre-built backward depacker is `exo-decrunch.bin` (in the `samples/` folder of C64 Visual Assembler) (477 bytes, ORG $B000). It's a Kick Assembler wrap of the official `exodecrunch.asm` with `INC $D020` added to every read for a visible border-flash effect during decompression. Place it into your program with `.incbin` at the depacker address.

**Raw input:** the extra is passed to Exomizer as a raw file loaded at the Dst address ( `file@<Dst>` ), so the depacker writes every byte, starting with the first one, exactly at the address you typed.

> **See also:** the `exo-multicolor-demo` sample — full end-to-end example: LOADFILE a compressed multicolor bitmap to $C000, EXODECRUNCH unpacks it to $2000, then copy screen → $0400 and color → $D800, and switch VIC-II to multicolor bitmap mode.

---

## REU_CHECK

Detects whether a Commodore RAM Expansion Unit (REU) is plugged in — like checking `PEEK($D010)` to see if hardware is present. Tests by writing and reading back two patterns to REU register `$DF04`.

| Argument | Description |
|---|---|
| None | No operands |

**Generated code (34 bytes):**

```
LDA #$55
STA $DF04
LDA $DF04
CMP #$55
BNE fail
LDA #$AA
STA $DF04
LDA $DF04
CMP #$AA
BNE fail
LDA #$00
CMP #$FF   ; Z=0 => REU present
BNE done
fail:
LDA #$FF
CMP #$FF   ; Z=1 => no REU
done:
```

> The macro normalizes the result so the following branch stays simple: `BNE` means REU present, `BEQ` means REU missing.

**Syntax:**

```
.reu_check
```

**Result in flags:**

- **Z = 0** (result ≠ 0) → REU present → use `BNE`
- **Z = 1** (result = 0) → no REU → use `BEQ`

**No arguments** — the macro generates the same code every time.

**Typical usage:**

```
.reu_check
BEQ no_reu        ; skip if REU not present
; ... REU code here ...
no_reu:
```

---

### REU_STASH / REU_FETCH / REU_SWAP

DMA block transfer between C64 RAM and REU expansion memory — like a very fast POKE loop, but the CPU doesn't do any work (the REU chip copies the data while the CPU is halted). A `$1000`-byte transfer is effectively instant.

| Macro | Direction | `$DF01` command |
|---|---|---|
| `REU_STASH` | C64 RAM → REU | `$90` |
| `REU_FETCH` | REU → C64 RAM | `$91` |
| `REU_SWAP` | C64 RAM ↔ REU | `$92` |

**Arguments:**

| Argument | Description | Example |
|---|---|---|
| C64 address | Source/dest in C64 RAM (hex) | `C000` |
| REU address | Source/dest in REU (hex, 16-bit) | `0000` |
| REU bank | REU memory bank (0–7) | `0` |
| Length | Number of bytes to transfer (hex, 16-bit) | `1000` |

**Syntax:**

```
.reu_stash $C000, $0000, 0, $1000
.reu_fetch $C000, $0000, 0, $1000
.reu_swap $C000, $0000, 0, $1000
```

**Generated code (40 bytes):**

```
LDA #c64lo    STA $DF02    ; C64 address LO
LDA #c64hi    STA $DF03    ; C64 address HI
LDA #reuLo    STA $DF04    ; REU address LO
LDA #reuHi    STA $DF05    ; REU address HI
LDA #bank     STA $DF06    ; REU bank
LDA #lenLo    STA $DF07    ; length LO
LDA #lenHi    STA $DF08    ; length HI
LDA #$00      STA $DF09    ; address control (fixed)
LDA #$00      STA $DF0A    ; interrupt mask (fixed)
LDA #cmd      STA $DF01    ; execute DMA ($90/$91/$92 = stash/fetch/swap, immediate)
```

> **Note:** Commands use `$90/$91/$92` (bit 4 set = immediate DMA mode). Writing to `$DF01` starts the transfer; the CPU resumes when it finishes.

---

TURBO_SET

Sets the **Ultimate-64 (U64) CPU speed** via register `$D031`. No effect on a real C64 or other emulators.

**Arguments:**

| Argument | Description | Range |
|---|---|---|
| Speed | CPU speed index | 0 = 1 MHz … 7 ≈ 10 MHz … 15 ≈ 48 MHz |
| Badline | Badline emulation | Enabled (C64 compatible) / Disabled (turbo) |

The speed byte is calculated as: `(speedIndex & 0x0F) | (badline_disabled ? 0x80 : 0x00)`.

**Generated code (5 bytes):**

```
A9 xx   LDA #speed_byte
8D 31 D0   STA $D031
```

**Syntax:**

```
.turbo_set 7,0    ; speed=7 (~10 MHz), badline enabled
.turbo_set 15,1   ; speed=15 (~48 MHz), badline disabled
```

> **Note:** This macro affects U64 hardware only. On a real C64 or other emulators this writes to `$D031` which may affect the CIA or be ignored.

---

SUPERCPU_DETECT

Checks whether a **CMD SuperCPU** accelerator is installed — like `PEEK($D0B8)` to see if it returns something other than `$FF`.

**Generated code (5 bytes):**

```
AD B8 D0   LDA $D0B8
C9 FF      CMP #$FF
```

**Result in flags:**

- **Z = 0** → SuperCPU present → use `BNE`
- **Z = 1** → SuperCPU not found → use `BEQ`

**No arguments.**

**Syntax:**

```
.supercpu_detect
```

**Typical usage:**

```
.supercpu_detect
BEQ no_scpu       ; skip if SuperCPU not present
```

```
    ; ... SuperCPU turbo code here ...
    no_scpu:
```

---

### TURBO_ENABLE

Turns **CMD SuperCPU turbo mode** on or off. Call `SUPERCPU_DETECT` first and skip this if the SuperCPU isn't present.

| Mode | Register | Effect |
|---|---|---|
| Enable | `$D07A` | Engage turbo (up to 20 MHz with SuperCPU) |
| Disable | `$D07B` | Return to 1 MHz compatibility mode |

**Generated code (5 bytes):**

```
A9 00         LDA #$00
8D 7A D0      STA $D07A    ; (or $D07B for disable)
```

**Syntax:**

```
.turbo_enable on
.turbo_enable off
```

> **Note:** Call `SUPERCPU_DETECT` first and branch around this macro if the SuperCPU is not present.

---

### MAP_COPY

Copies a tilemap from a source address to screen RAM (and optionally color RAM) using a series of `LDA abs,X` / `STA abs,X` loops. One 256-byte page is copied per loop iteration; a partial page at the end uses `CPX #rem / BNE` to stop. No JSR needed — all code is generated inline.

| Argument | Description |
|---|---|
| Source addr (screen) | Hex address where the map data lives after loading (e.g. `C000`) |
| Screen RAM dest | Where to copy screen codes (e.g. `0400`) |
| Size (bytes) | Total bytes to copy — typically `$03E8` = 1000 (40×25 chars) |

| Argument | Description |
|---|---|
| Color source | `auto`: the color data follows the screen codes ( `source + size` ); or an explicit color source address. The color data is copied to the color RAM destination in a second pass. Omit it (and the color destination) to copy screen codes only |
| Color RAM dest | Destination for color data — default `D800` (C64 color RAM) |

**Generated code (1000-byte map, screen only):**

```
LDX #$00
LDA $C000,X   ; page 0
STA $0400,X
INX
BNE *-9       ; loops until X wraps to 0 (256 iters)
LDA $C100,X   ; page 1
STA $0500,X
INX
BNE *-9
LDA $C200,X   ; page 2
STA $0600,X
INX
BNE *-9
LDA $C300,X   ; remainder — 232 bytes
STA $0700,X
INX
CPX #$E8
BNE *-11
```

**Size:** `2 (LDX) + fullPages×9 + (rem > 0 ? 11 : 0)` bytes per section. Combined mode doubles it (screen section + identical color section).

**Pairing with the Map Editor:**

The Map Editor of C64 Visual Assembler (**Save map + color RAM**) exports a single binary where the first `size` bytes are screen codes and the next `size` bytes are color RAM values. Use `.map_copy` with `auto` as the color source and the source address where this file is loaded (e.g. with `.incbin` at `$C000`):

```
.incbin "map-color.bin", $C000   ; screen codes $C000–$C3E7, color $C3E8–$C7CF
; ...
.map_copy $C000, $0400, 1000, auto, $D800
```

**Syntax:**

```
.map_copy $C000, $0400, 1000               ; screen only
.map_copy $C000, $0400, 1000, auto, $D800  ; combined (color at src+size)
.map_copy $C000, $0400, 1000, $C3E8, $D800 ; explicit color source address
```

---

MAP_COPY16X16

Copies a 16×16 character area from a compact 256-byte screen-code block plus a matching 256-byte Color RAM block. It is intended for Charset Canvas exports and small tile/picture chunks, where writing sixteen separate MAP_COPY rows would be noisy.

**Default layout:**

| Data | Default address |
|---|---|
| 16×16 screen codes | Source address ( `src` ) |
| 16×16 color values | `src + 256` |
| Screen RAM destination | `$0400 + row×40 + col` |
| Color RAM destination | `$D800 + row×40 + col` |

**Syntax:**

```
.map_copy16x16 $3000, 12, 4
.map_copy16x16 $3000, 12, 4, $0400, $3100, $D800
```

The short form copies screen bytes from `$3000`, color bytes from `$3100`, and places the 16×16 block at column 12, row 4. Valid top-left positions are `col = 0..24` and `row = 0..9`, so the full 16×16 area remains on the 40×25 C64 text screen.

**Generated behavior:**

- Generates sixteen inline row copies.
- Each row copies 16 screen bytes and 16 color bytes.
- No JSR is needed; the code is emitted directly at the macro position.
- Works with normal character mode and multicolor character mode; the Color RAM bytes carry each cell's character color / multicolor enable bit.

Typical pairing with Charset Canvas:

```
* = $0801
    SEI
    LDA #$00
    STA $D021
    LDA #$05
    STA $D022
    LDA #$0D
    STA $D023
    .map_copy16x16 $3000, 12, 4
loop:
    JMP loop

.incbin "16x16+ram+color.bin", $3000
.incbin "16x16+charset.bin", $2000
```

---

SPRITE_ANIM

Advances a sprite's animation frame each call and updates the VIC-II sprite data pointer. Store one byte per frame in a table (the sprite data page number = `data_address / 64`), point SPRITE_ANIM at it, and call it once per game frame — no JSR needed.

| Argument | Description |
|---|---|
| Sprite # | Sprite number 0–7 |
| Frame list address | Hex address of the frame table — one byte per frame, each byte = sprite page ( `data_addr / 64` ) |
| Frame count | Total number of frames (1–255) |
| Frame ZP | Zero-page byte used as the frame counter (e.g. `FB` ) |

**Generated code (sprite 0, 4 frames, ZP `$FB`, list at `$C100`):**

```
    INC $FB         ; advance frame counter
    LDA $FB
    CMP #$04        ; frame count
    BCC *+6         ; if counter < count, skip reset
    LDA #$00
    STA $FB
    TAX             ; X = current frame index
    LDA $C100,X     ; load sprite data page for this frame
    STA $07F8       ; update sprite 0 pointer ($07F8 + sprite#)
```

**Size:** 19 bytes.

**Typical usage:**

```
frameTable:
    .byte $21, $22, $23, $24   ; 4 frames at $0840, $0880, $08C0, $0900

gameloop:
    .wait_raster $FF
    .sprite_anim 0, $C100, 4, $FB
    JMP gameloop
```

**Syntax:**

```
.sprite_anim spriteNum, frameListAddr, frameCount, zpByte
; example:
.sprite_anim 0, C100, 4, FB
```

> **Tip:** Place the frame table with `.rawbytes` at a fixed address. The counter ZP byte ( `$FB` ) must be initialized to `$00` before the first call. If your code uses `$FB` for something else, pick a free ZP location.

---

SCORE_BCD

Adds a fixed point value to a multi-byte BCD score stored in memory, then renders each digit to screen RAM as a screen-code character. Uses the 6502 decimal mode ( `SED` / `CLD` ) for carry-safe BCD arithmetic — no manual carry juggling needed.

| Argument | Description |
|---|---|
| Score address | Hex address of the BCD score bytes (e.g. `C200` ). Low byte first. |
| Digits | Number of BCD bytes (each byte holds two digits: `$99` = "99"). `4` bytes = up to 99999999. |
| Add points | Decimal value to add per call (e.g. `100` ). |
| Screen address | Where to write the digit screen codes (e.g. `0400` ). One byte per digit (high nibble first). |

**Syntax:**

```
.score_bcd $C200, 4, 100, $0400
```

**Generated code (4 bytes, +100 pts, score at `$C200`, screen at `$0400`):**

```
      SED             ; enable BCD / decimal mode
      CLC
      LDA $C200       ; byte 0 (digits 1–2)
      ADC #$00        ; low byte of 100 in BCD = $00
      STA $C200
      LDA $C201       ; byte 1 (digits 3–4)
      ADC #$01        ; mid byte of 100 in BCD = $01 (carry propagates)
      STA $C201
      LDA $C202
      ADC #$00
      STA $C202
      LDA $C203
      ADC #$00
      STA $C203
      CLD             ; back to binary mode

      ; render digits
      LDX #$00
  digit_loop:
      LDA $C200,X
      PHA
      LSR : LSR : LSR : LSR  ; high nibble → low nibble
      ORA #$30        ; + '0' screen code
      STA $0406,X     ; right-justified: screen_addr + (digits*2 - 2) - X*2
      PLA
      AND #$0F        ; low nibble
      ORA #$30
      STA $0407,X
      INX
      CPX #$04
      BNE digit_loop
```

**Size:** `3 + digits×8` bytes (SED + CLC + CLD overhead + 8 bytes per BCD byte for ADC + display loop).

**Syntax:**

```
  .score_bcd $C200, 4, 100, $0400
```

> **Tip:** Initialize the score bytes to `$00` at startup. The score address should be in zero page or absolute RAM — not ROM. The screen address should point to the leftmost digit cell; digits are written left-to-right (most-significant byte first).

> **BCD range:** `digits=4` bytes → 8 decimal digits → max score 99,999,999. Each byte encodes two BCD digits: `$00` – `$99`.

## 7. KickAssembler-style syntax

VMA reads the common KickAssembler forms too, so a KickAssembler source often builds as it is:

- `.pc = $1000` and `*=$1000 "name"` set the address (the segment name is ignored).
- `.const NAME = value` and `.label NAME = value` define constants; `.label NAME = *`, `.label NAME = * + 1` puts a
  label at (or near) the current address.
- `.macro name(p1, p2) { … }` — a macro body in braces, besides `.macro … .endm`; `:name(args)` and `name(args)` call it.
- `name: { … }` scopes: the labels inside are local to `name` (`name.label` from outside).
- `@label` labels keep their `@`.
- `.text "…"` without screen coordinates stores the encoded bytes like `.byte`. Several items may be joined with `,`
  or `+`: `"…"`, `@"…"` with the escapes `\n` `\r` `\t` `\$NN`, and `$NN` / decimal bytes. `.encoding "name"` selects the encoding of the following `.text` lines: `screencode_mixed` (default), `screencode_upper`, `petscii_mixed`,
  `petscii_upper`, `ascii`; a character the encoding has no code for is an error.
- Mnemonic suffixes `.abs` / `.a` and `.zp` / `.z` force absolute or zero-page addressing: `sta.abs $00FC,x`, `ldx.zp ptr,y`.
- `// …` comments.

The rest of the KickAssembler language (`.for`, `.var` / `.eval`, `.function`, `.pseudocommand`, `#import`, `#if`, `LoadBinary`, segments …) is handled by the Kick preprocessor of C64 Visual Assembler, which turns a KickAssembler source tree into VMA text.

## 8. Command line

New in 2.1.0: `--crt` / `--crt-name` (cartridge) and `--d64` / `--add` / `--disk-name` (disk image).

```
vma build <input.asm> [options]
vma --help
vma --version
```

| Option | Meaning |
|---|---|
| `-o FILE` | The PRG to write (default: the input with a `.prg` extension) |
| `--base-dir DIR` | Where `.incbin` / `.koala` / `.level` / `.sid` / `.include` files are looked up first (default: the folder of the input) |
| `--search DIR` | A further folder to look in, after the base dir (repeatable, in order); a path written on another machine falls back to its bare file name |

| Option | Meaning |
| --- | --- |
| `--no-basic-stub` | No `10 SYS…` BASIC line in front of the code |
| `--raw` | The image alone at its first ORG: no stub, no move to `$C000` |
| `--crt FILE` | A Magic Desk cartridge (CRT type 19, 8 × 8K banks, at most 65 408 bytes): a loader copies the program to its load address and starts it at the SYS address — the Visual Assembler's Build CRT. Without `-o` no PRG is written |
| `--crt-name NAME` | The cartridge name in the CRT header (default: `VA CART`) |
| `--d64 [FILE]` | A 1541 disk image with the PRG on it, named after the PRG (default FILE: the PRG with a `.d64` extension); no c1541 needed. Without `-o` no PRG is written |
| `--add FILE` | A further file on the disk as it is (a PRG with its load address), named after the file (repeatable, in order) |
| `--disk-name NAME` | The disk name (default: the D64's file name; ID `01`) |
| `--sym FILE` | KickAssembler label file (`.label NAME=$xxxx`) |
| `--vs FILE` | VICE monitor label file (`al C:xxxx .NAME`; `x64sc -moncommands FILE`) |
| `--dbg FILE` | RetroDebugger / C64 Debugger source map (XML) |
| `--debug` | All three next to the PRG (`out.sym`, `out.vs`, `out.dbg`) |
| `--json FILE` | The build as JSON: PRG, SYS address, symbols, laid-out lines, written segments, memory view — or every error with its line |
| `--source-path NAME` | The main source the `.dbg` names (default: the input) |
| `--line-map FILE` | JSON array: input line → the line the `.dbg` names |

Exit status: `0` built, `1` compile error (`FILE:LINE: message` on standard error, one line per error), `2` bad usage.

```
vma build game.asm -o game.prg --debug
vma build game.asm --no-basic-stub --search ../assets
vma build game.asm --crt game.crt                   # the cartridge only
vma build game.asm --d64 --add music.prg            # game.d64 with GAME and MUSIC
```

## 9. Known limitations

VMA reproduces the C64 Visual Assembler compiler byte for byte, its rough edges included:

- `lda.zp c` → `LDA $0C`: with `.zp` a symbol made of hex digits is read as a number.

- The body of a parameter-less macro is also assembled where it is defined; a definition before the first `* =` lands at the start of the program.
- `.text X, Y, "s", 80` parses the character offset but does not use it.
- The bytes of `.chardef` are hex (`129` = `$129`, an error).
- The non-immediate operand of a runtime `.if` / `.while` / `.until` is read as an address: `%bin` is an error and `10` is hex (`$0010`).
- The DELAY helper routine goes after the last line of the layout and every data section — even for a skipped (`.if 0`) DELAY.

Fixed in 2.1 (they gave wrong code or failed before): a `.const` / `.var` operand in the zero page, `JMP (label)`, `lda $1234+1`, a macro call inside a macro, an unresolved TURBO_SET / SPRITE_ANIM value (now an error), labels in the bytes of `.data` / `.rawbytes`, a `:name` label whose target is a label, the size of JOYSTICK / MEMCPY / MEMSET from a later label (now an error), the automatic buffer of `.data` / `.rawbytes`, and the includes (a missing one is an error, an included file's own `.include` / `.incbin` / `.sid` are loaded).